Ôn phỏng vấn Fullstack

React / Frontend · Kiến trúc Backend Node.js · Gắn với dự án Team Task Management · 27/10/2026

> **Khung trả lời một câu hỏi ở mức senior:** (1) Trả lời thẳng bằng 1–2 câu. (2) Giải thích **vì sao**, tức cơ chế bên dưới. (3) Nêu **trade-off**: khi nào nên dùng, khi nào không. (4) Đưa **ví dụ thật** từ dự án. (5) Nói cách **đo hoặc kiểm chứng**. Người phỏng vấn senior chấm chủ yếu ở bước 2 đến 5, không chấm định nghĩa.

Phần A: Trả lời 16 câu hỏi phỏng vấn **Phần B**: Lý thuyết FE và tối ưu hiệu năng **Phần C**: Cấu trúc và lộ trình Backend

Phần A: Trả lời các câu hỏi phỏng vấn

A1. React nền tảng

1 Props và state khác nhau thế nào?

> **Ngắn:** **props** là dữ liệu cha truyền xuống, component con chỉ đọc. **state** là dữ liệu component tự sở hữu và tự thay đổi qua `setState`. Cả hai thay đổi đều khiến component render lại.

	props	state
Ai sở hữu	Component cha	Chính component đó
Có được đổi không	Không (read-only)	Có, qua setter, và phải tạo object mới (immutable)
Mục đích	Cấu hình component từ bên ngoài	Ghi nhớ dữ liệu thay đổi theo thời gian, ví dụ input hay trạng thái mở/đóng

> **Bẫy: Không copy props vào state** (`useState(props.user)`). State chỉ lấy giá trị ở lần mount đầu, nên khi props đổi thì state không cập nhật theo. Nếu giá trị tính được từ props thì **tính trực tiếp khi render** (derived state). Nếu thật sự cần reset state khi props đổi thì đổi `key` của component.

2 Component khai báo state nhưng không dùng tới. Khi props đổi hoặc cha render lại, nó có render lại không?

> **Ngắn: Có.** Mặc định, khi cha render thì **mọi con đều render lại**, bất kể props có đổi hay không, và bất kể con có state hay không. Việc khai báo state không ảnh hưởng gì tới chuyện này.

Có 3 lý do khiến một component render lại:

1. State của **chính nó** thay đổi (gọi setter với giá trị mới, so sánh bằng `Object.is`).
2. **Cha render lại.**
3. Một **context** mà nó dùng (`useContext`) đổi giá trị.

Muốn bỏ qua trường hợp (2) thì bọc component bằng `React.memo`. Khi đó React so sánh nông (shallow) từng prop, nếu tất cả bằng nhau thì bỏ qua render. Vì vậy các prop dạng object hay function phải giữ **cùng tham chiếu** giữa các lần render, bằng `useMemo` hoặc `useCallback`. Nếu không, `memo` vô tác dụng.

> **Ý senior:** "render lại" chỉ là gọi lại hàm component và so sánh Virtual DOM. Nó **không** có nghĩa là DOM bị cập nhật. React chỉ ghi vào DOM những gì thay đổi (giai đoạn commit). Render lại không xấu, chỉ đáng lo khi nó **tốn kém và đo được**. React Compiler (bản 1.0 ra tháng 10/2025) tự động memo hoá, nên giảm nhu cầu viết `useMemo` / `useCallback` bằng tay.

3 Key trong React là gì? Đặt key thế nào? Hai list userA và userB cùng dùng index làm key, khi chuyển trang thì có ảnh hưởng gì không?

> **Ngắn:** `key` giúp React biết phần tử nào trong danh sách là "cùng một phần tử" giữa 2 lần render, để **giữ lại** component instance và state của nó. Key chỉ cần **duy nhất trong cùng một danh sách anh em** (siblings), không cần duy nhất toàn app. Nên đặt key bằng **id ổn định từ dữ liệu**, ví dụ `user.id`.

- **Hai list khác nhau, mỗi list dùng index riêng: không xung đột với nhau**, vì key chỉ được so sánh trong phạm vi một list. Ngoại lệ: nếu nối 2 list vào **cùng một mảng con** (`[...a.map(), ...b.map()]`) thì sẽ bị trùng key 0, 1, 2. React sẽ cảnh báo và có thể dùng nhầm phần tử.
- **Dùng index làm key khi chuyển trang (pagination):** dòng đầu tiên ở trang 2 có key `0`, giống hệt dòng đầu ở trang 1. React coi đó là **cùng một component**, nên **giữ lại state cũ**: ô checkbox đang tick, dòng đang mở rộng, giá trị input đang gõ dở. Kết quả là user ở trang 2 "thừa hưởng" trạng thái của user ở trang 1.
- Tương tự khi **chèn vào đầu, xoá hoặc sắp xếp lại**: state bị lệch sang dòng khác, và React phải render lại mọi dòng phía sau.
- Index chỉ chấp nhận được khi danh sách **tĩnh**: không bao giờ đổi thứ tự, không thêm hay xoá, và từng dòng không có state.

> **Bẫy:** `key={Math.random()}` khiến mỗi lần render tạo key mới. React sẽ huỷ và mount lại toàn bộ phần tử, vừa chậm vừa mất state.

4 StrictMode trong React dùng để làm gì?

> **Ngắn:** `<StrictMode>` là công cụ **chỉ chạy ở môi trường dev**, giúp phát hiện code không "thuần" (impure). Nó không render ra UI gì và không ảnh hưởng tới production.

- **Gọi hàm render 2 lần** (thân component, initializer của `useState`, `useMemo`...) để lộ ra side effect nằm sai chỗ trong lúc render.
- **Chạy effect theo chu kỳ mount, unmount, rồi mount lại** để lộ ra effect **thiếu cleanup**, ví dụ đăng ký 2 lần listener hoặc mở 2 lần WebSocket.
- Cảnh báo khi dùng các API đã lỗi thời.

> **Câu hỏi follow-up hay gặp:** "Vì sao `useEffect` gọi API 2 lần?" Đó là do StrictMode ở dev. Cách xử lý đúng là viết cleanup (`AbortController`) hoặc dùng TanStack Query, **không phải** tắt StrictMode.

5 Khi nào dùng state, khi nào dùng store, khi nào dùng context?

Loại dữ liệu	Công cụ	Ví dụ
Chỉ một component cần	`useState` / `useReducer`	Dialog mở hay đóng, ô input
Vài component gần nhau cần	**Lift state** lên cha chung gần nhất	Tab và nội dung của tab
Giá trị **ít thay đổi**, cần truyền sâu xuống nhiều tầng	**Context**	Theme, ngôn ngữ, user hiện tại, dependency injection
Client state **thay đổi thường xuyên**, dùng ở nhiều nơi, hoặc cần đọc từ **ngoài React**	**Store** (Zustand, Redux)	Auth token cho axios interceptor, giỏ hàng, bộ lọc phức tạp
Dữ liệu lấy từ server	**TanStack Query** (không phải store)	Danh sách workspace, chi tiết task
Bộ lọc, trang, tab muốn chia sẻ được qua link	**URL** (search params)	`?page=2&status=DONE`
Form	react-hook-form	Form tạo workspace

Ý senior: Context **không phải công cụ quản lý state**, nó chỉ là cơ chế **truyền dữ liệu** xuống cây component. Khi `value` đổi, **mọi component dùng context đó đều render lại**, kể cả khi chúng chỉ đọc một phần nhỏ. Store như Zustand có **selector**: `useStore(s => s.user)` chỉ render lại khi đúng phần `user` thay đổi.

Dự án: auth được đặt trong Zustand vì axios interceptor và BroadcastChannel (code ngoài React) cần đọc và ghi token. Dữ liệu workspace thì nằm trong TanStack Query.

6 So sánh Promise và async/await

Ngắn: `async/await` là **cú pháp viết trên nền Promise**, không phải một cơ chế mới. Hàm `async` luôn trả về một Promise. `await` tạm dừng **hàm đó** (không chặn luồng chính), và phần còn lại của hàm được đưa vào microtask queue để chạy sau.

```
// Promise chain
getUser(id)
  .then(u => getOrders(u.id))
  .then(orders => render(orders))
  .catch(handleError)
  .finally(stopLoading);
```

```
// async/await: đọc như code tuần tự
try {
  const u = await getUser(id);
  const orders = await getOrders(u.id);
  render(orders);
} catch (e) { handleError(e); }
finally { stopLoading(); }
```

Bẫy hay gặp:

- **await tuần tự khi các việc không phụ thuộc nhau:** `await a(); await b();` mất thời gian bằng a + b. Dùng `await Promise.all([a(), b()])` thì chỉ mất bằng thời gian của việc lâu nhất.
- **`forEach` với hàm async** không chờ các lần gọi xong. Dùng `for...of` (chạy tuần tự) hoặc `Promise.all(arr.map(...))` (chạy song song).
- **Quên `await`** thì lỗi trở thành *unhandled rejection* và không bị `try/catch` bắt.
- Biết thêm: `Promise.all` fail ngay khi có 1 promise lỗi. `allSettled` chờ tất cả xong. `race` lấy kết quả về đầu tiên. `any` lấy kết quả thành công đầu tiên.

A2. Kiến trúc component và UI

7 Component B nằm trong A. Muốn bấm nút ở B để mở modal, nhưng modal lại nằm ở A?

Ngắn: Lift state up. State `open` đặt ở A, A truyền callback `onOpen` xuống B. Nếu B nằm sâu nhiều tầng thì dùng context hoặc một store "modal manager".

```
function A() {
  const [open, setOpen] = useState(false);
  return (<>
    <B onOpenModal={() => setOpen(true)} />
    <Modal open={open} onClose={() => setOpen(false)} />
  </>);
}
```

- **B nằm sâu nhiều tầng:** dùng Context `{ openModal }`, hoặc store `useModalStore.open("confirm", props)` kèm một `<ModalRoot/>` duy nhất ở App. Đây là pattern hay dùng cho confirm dialog và toast toàn app.
- **Phân biệt với Portal:** `createPortal(modal, document.body)` chỉ quyết định **vị trí của DOM** (để thoát khỏi `overflow:hidden` hay `z-index` của cha). Còn về logic, component vẫn nằm trong cây React của B, sự kiện vẫn nổi bọt (bubble) theo cây React. Các thư viện Dialog như Base UI hay Radix đã tự dùng portal sẵn.
- `useImperativeHandle` (A gọi `modalRef.current.open()`) cũng là một cách, nhưng kém "React", chỉ nên dùng khi bọc thư viện bên ngoài.

Dự án: `WorkspaceListPage` giữ state `openDialog` và truyền `onClose` xuống `CreateWorkspaceDialog`. Đây chính là lift state.

8 Setup biến theme (light/dark) như thế nào?

1. **Design token bằng CSS variables**: khai báo `--background`, `--primary`... trên `:root`, và khai báo lại giá trị khác dưới `[data-theme="dark"]` (hoặc class `.dark`). Component chỉ dùng token, không bao giờ dùng mã màu trực tiếp.
2. **Giá trị mặc định** lấy theo hệ điều hành: `@media (prefers-color-scheme: dark)`.
3. **Lựa chọn của user** được lưu vào `localStorage`. Nút toggle đổi thuộc tính `data-theme` trên thẻ `<html>`, thông qua một context hoặc store nhỏ.
4. **Chống nháy màu (FOUC):** đặt một **script inline nhỏ trong `<head>`**, đọc localStorage và gắn `data-theme` **trước khi trang được vẽ**. Nếu gắn trong `useEffect` thì trang sẽ hiện màu sáng một nhịp rồi mới chuyển sang tối.

Đổi theme bằng CSS variable thì **không làm React render lại**, vì chỉ đổi một thuộc tính trên `<html>`, trình duyệt tự tính lại style. Cách này nhanh hơn nhiều so với truyền object màu qua context.

9 Trình bày về design system mà bạn biết

> **Ngắn:** Design system là **nguồn chuẩn duy nhất** cho UI, gồm token, component, pattern, tài liệu và quy trình. Mục tiêu là UI nhất quán, làm nhanh hơn, và mặc định đảm bảo accessibility.

Tầng	Nội dung
Design tokens	Màu (theo vai trò: `primary`, `destructive`, `muted`), spacing, typography, radius, shadow. Thường lưu dưới dạng CSS variables hoặc JSON, đồng bộ với Figma.
Primitives / components	Button, Input, Dialog, Select... có **variants** (ví dụ `cva`: `variant`, `size`). Hành vi và a11y lấy từ **headless library** (Radix, Base UI, React Aria), phần style tự làm.
Patterns	Form field có label và lỗi, empty state, skeleton, data table.
Tài liệu và kiểm thử	Storybook, visual regression test, a11y test (axe).
Quản lý (governance)	Versioning (semver), changelog, quy trình đóng góp, deprecation.

Các ví dụ nên biết: Material UI (theo Material Design), Ant Design (cho admin, dashboard), IBM Carbon, và **shadcn/ui**. shadcn/ui không phải thư viện cài qua npm, mà là code component được copy thẳng vào dự án để bạn sở hữu và tự sửa.

> **Dự án:** `ui/src/shared/ui` theo phong cách shadcn, dùng Base UI làm headless, token là CSS variables. `FormField` tự gắn `aria-invalid` và `aria-describedby`. Đây là ví dụ cụ thể của "a11y được làm sẵn trong design system".

A3. Hiệu năng

10 Người dùng mở nhiều tab trang bảng giá (realtime). Làm sao tối ưu hiệu năng?

> **Vấn đề:** mỗi tab tự mở một WebSocket, tự parse message, tự render. Mở N tab thì tốn N kết nối, server tốn N lần băng thông, máy tốn N lần CPU, trong khi user chỉ nhìn được **một** tab tại một thời điểm.

(a) Dùng chung một kết nối cho mọi tab

- **SharedWorker** giữ một WebSocket duy nhất và gửi dữ liệu cho các tab qua `port.postMessage`.
- Hoặc **bầu tab leader** bằng `navigator.locks.request("price-leader", ...)` (Web Locks API). Tab leader mở WebSocket, rồi phát dữ liệu cho các tab khác qua **BroadcastChannel**. Khi tab leader đóng, lock được giải phóng và một tab khác tự lên làm leader.

(b) Tab bị ẩn thì không làm việc

- Dùng **Page Visibility API** (`document.visibilityState`, sự kiện `visibilitychange`): tab bị ẩn thì huỷ đăng ký nhận dữ liệu hoặc ngừng render. Khi tab hiện lại thì lấy một snapshot mới nhất.

(c) Render ít nhất có thể, đây là phần quan trọng nhất trong một tab

- **Gom các update theo frame:** message đến liên tục thì ghi vào buffer, mỗi `requestAnimationFrame` (hoặc mỗi 100–250ms) mới flush một lần, thay vì gọi `setState` cho từng message.
- **Chỉ render ô thay đổi:** lưu giá trị theo từng mã chứng khoán trong store ngoài React (Zustand, hoặc `useSyncExternalStore`). Mỗi dòng dùng selector `useStore(s => s.prices[symbol])`, nên một mã thay đổi chỉ khiến 1 dòng render lại, không phải cả bảng.
- Bọc dòng bằng `React.memo` với `key={symbol}`. Nếu bảng dài thì dùng **virtualization**.
- Hiệu ứng nháy xanh/đỏ khi giá đổi: làm bằng CSS class hoặc animation, không lưu thêm state.

(d) Phía server: chỉ gửi **phần thay đổi** (diff), không gửi toàn bộ bảng. Client chỉ đăng ký những mã đang hiển thị. Nén message (permessage-deflate) hoặc dùng định dạng nhị phân.

> **Dự án:** auth đã dùng BroadcastChannel để đồng bộ đăng xuất giữa các tab. Đây đúng là cơ chế ở ý (a).

11 API trả về bảng có rất nhiều user. FE tối ưu thế nào?

> **Nguyên tắc:** cách tối ưu tốt nhất là **không tải và không render những gì user không nhìn thấy**. Ưu tiên theo thứ tự: giảm dữ liệu, rồi giảm số DOM, rồi mới giảm số lần render.

1. **Phân trang, lọc, sắp xếp phía server**: `?page=&limit=&sort=&q=`. Không bao giờ tải 50.000 dòng về client. Khi bảng có hàng triệu dòng thì dùng phân trang theo cursor.
2. **Chỉ lấy các cột cần thiết**: API trả về DTO gọn (dùng `select` phía BE).
3. **Virtualization** (TanStack Virtual, react-window): chỉ đưa vào DOM khoảng 20–40 dòng đang nằm trong khung nhìn, cho dù có 10.000 dòng.
4. **Infinite scroll**: `useInfiniteQuery` kết hợp `IntersectionObserver` để tải trang tiếp theo khi cuộn gần cuối.
5. **Cache và chuyển trang mượt**: dùng TanStack Query với `placeholderData: keepPreviousData` để giữ trang cũ trong lúc tải trang mới, không nhấp nháy. Có thể `prefetchQuery` trang kế tiếp trước.
6. **Dòng ổn định**: `key={user.id}`, bọc `React.memo(Row)`, callback dùng `useCallback`. Dữ liệu dẫn xuất (đã lọc, đã sắp xếp) tính bằng `useMemo`, không lưu thêm vào state.
7. **Ô tìm kiếm**: dùng debounce. Nếu lọc phía client thì dùng `useDeferredValue` để việc gõ phím không bị lag.
8. **Xử lý nặng** (sắp xếp hay biến đổi hàng chục nghìn dòng, xuất CSV): đưa vào **Web Worker** để không chặn luồng chính.

12 Component search và chọn user: làm sao cho mượt?

Vấn đề	Giải pháp
Mỗi phím gõ gọi một API	**Debounce** khoảng 300ms, và chỉ tìm khi đã gõ từ 2 ký tự trở lên

Race condition: response của "ab" về sau response của "abc"	Huỷ request cũ bằng `AbortController`. TanStack Query truyền sẵn `signal` và luôn lấy kết quả của query key mới nhất.
Gõ lại từ đã tìm vẫn gọi API	Cache theo key `["users", q]`
Danh sách nhấp nháy khi chờ	`placeholderData: keepPreviousData`, và hiện loading nhỏ trong ô input
Hàng nghìn lựa chọn	Virtualize danh sách option, hoặc chỉ trả về tối đa 20 kết quả
Lọc phía client làm gõ phím bị lag	`useDeferredValue(query)` hoặc `useTransition`
Accessibility	Đúng pattern **combobox**: `role="combobox"`, `aria-expanded`, `aria-activedescendant`, điều khiển bằng phím mũi tên, Enter, Esc. Nên dùng component Combobox headless có sẵn.
Backend tìm kiếm chậm	Đánh index cho cột tìm kiếm. Với `ILIKE '%x%'` trên Postgres thì dùng trigram index (`pg_trgm`).

13 useTransition là gì?

> **Ngắn:** dùng để đánh dấu một cập nhật state là **không gấp**. React ưu tiên các cập nhật gấp (gõ phím, click), và có thể **tạm dừng hoặc bỏ dở** việc render của cập nhật không gấp để giữ UI phản hồi nhanh. Hook trả về `isPending` để hiển thị trạng thái chờ.

```
const [isPending, startTransition] = useTransition();
const onChange = (e) => {
  setText(e.target.value);                          // gấp: ô input cập nhật ngay
  startTransition(() => setFilter(e.target.value)); // không gấp: lọc danh sách lớn
};
```

- **So với debounce:** debounce **trì hoãn** một khoảng thời gian cố định. Transition không có độ trễ cố định: máy nhanh thì cập nhật gần như ngay, máy chậm thì vẫn mượt vì việc render có thể bị ngắt. Nhưng transition **không giảm số request**, nên khi gọi API vẫn cần debounce.
- **`useDeferredValue(value)`** có cùng ý tưởng, dùng khi bạn chỉ nhận được **giá trị** (ví dụ từ props), không kiểm soát được hàm setter.
- Không dùng transition cho chính giá trị của controlled input, vì input sẽ bị lag.
- Từ React 19, `startTransition` nhận được hàm async (gọi là Actions), kết hợp với `useActionState` và `useOptimistic`.

14 Ví dụ về memory leak khi làm React, và cách khắc phục

Nguyên nhân	Cách sửa
`setInterval` hoặc `setTimeout` không được huỷ	Trả về hàm cleanup trong effect: `return () => clearInterval(id)`
`window.addEventListener("resize")` không được gỡ	`removeEventListener` với **đúng tham chiếu hàm** đã đăng ký
WebSocket, subscription của store hay observable không đóng	`ws.close()` hoặc `unsubscribe()` trong cleanup
Request vẫn chạy và gọi `setState` sau khi component đã unmount	`AbortController` và gọi `abort()` trong cleanup, hoặc dùng TanStack Query
Instance của thư viện bên thứ 3 (chart, map, editor) không được huỷ	Gọi `chart.destroy()` trong cleanup
Cache hoặc Map ở cấp module tăng mãi không giới hạn	Giới hạn kích thước (LRU), hoặc dùng `WeakMap`
Closure giữ dữ liệu lớn trong một listener sống lâu	Gỡ listener, và không giữ tham chiếu tới DOM đã bị gỡ

Cách phát hiện: mở Chrome DevTools, tab **Memory**, chụp heap snapshot trước và sau khi mở rồi đóng một màn hình vài lần, sau đó so sánh. Lọc theo "Detached" để tìm DOM đã bị gỡ nhưng vẫn bị giữ lại. Bật StrictMode để sớm lộ ra các

effect thiếu cleanup.

A4. Web và mạng

15 WebSocket là gì? Làm sao client B nhận được message do client A gửi?

> **Ngắn:** WebSocket là kết nối **hai chiều (full-duplex), giữ mở lâu dài** giữa client và server. Nó bắt đầu bằng một request HTTP có header `Upgrade: websocket`, sau đó cả hai bên đều có thể chủ động gửi dữ liệu bất cứ lúc nào, không cần mô hình request/response.

```
Client A ──send("hi", to: room 42)──► SERVER ──tìm các socket trong room 42──► Client B
                                      │  Map<userId|room, Set<socket>>
                                      └─ (nhiều instance) Redis Pub/Sub ──► server khác ──► Client C
```

- A và B **không nói chuyện trực tiếp với nhau**. Cả hai cùng kết nối tới server. Server giữ bảng ánh xạ user hoặc room tới các socket, nhận message từ A rồi **chuyển tiếp** tới các socket của B. Kết nối peer-to-peer trực tiếp là chuyện của WebRTC.
- **Room** (Socket.IO): ví dụ client tham gia room `board:42`, server phát message cho room đó.
- **Khi chạy nhiều instance server**: A kết nối vào server 1, B kết nối vào server 2. Cần một **Redis adapter (Pub/Sub)** để các server chuyển message cho nhau.
- **Thực tế production cần thêm:** xác thực ngay lúc handshake (token), heartbeat (ping/pong), tự kết nối lại với backoff, lấy lại dữ liệu bị lỡ khi kết nối lại.
- **Lựa chọn khác:** nếu chỉ cần server đẩy dữ liệu xuống (thông báo, bảng giá) thì **SSE** đơn giản hơn, chạy trên HTTP và có sẵn tự kết nối lại.

> **Dự án (Task 10, Kanban):** khi user A kéo một task sang cột khác, server phát sự kiện tới room `board:id`. Client B nhận sự kiện và gọi `queryClient.invalidateQueries` (hoặc `setQueryData`) để cập nhật bảng.

16 Phân biệt session, cookie và localStorage

	Cookie	Session (phía server)	localStorage	sessionStorage
Lưu ở đâu	Trình duyệt	**Server** (bộ nhớ, Redis, DB). Trình duyệt chỉ giữ **session id**, thường nằm trong cookie	Trình duyệt	Trình duyệt, **riêng từng tab**
Dung lượng	Khoảng 4KB mỗi cookie	Không giới hạn đáng kể	Khoảng 5–10MB mỗi origin	Khoảng 5MB
Tự gửi kèm request	**Có**, với mọi request tới domain đó	Gửi qua cookie chứa id	Không	Không
JavaScript đọc được	Có, **trừ khi đặt `HttpOnly`**	Không	Có	Có
Hết hạn	`Expires` / `Max-Age`	Do server quyết định	Không bao giờ	Khi đóng tab
Rủi ro	CSRF (giảm bằng `SameSite`)	Phải chia sẻ store khi scale nhiều server	**XSS đọc được**	XSS

> **Dự án:** refresh token đặt trong cookie `HttpOnly; Secure; SameSite`, nên JS không đọc được và XSS không lấy được. Access token chỉ giữ trong **bộ nhớ** (Zustand), không lưu localStorage. Khi F5 trang, app gọi `/auth/refresh` để lấy access token mới.

Phần B: Lý thuyết Frontend và tối ưu hiệu năng

B1. Cách React render: phải hiểu thì mới tối ưu được

```
Trigger (setState / cha render / context đổi)
   └─► RENDER: gọi hàm component → tạo cây element mới → so sánh với cây cũ (reconciliation, dựa trên type + key)
         └─► COMMIT: chỉ ghi phần khác biệt vào DOM → chạy useLayoutEffect → trình duyệt vẽ → chạy useEffect
```

- **Batching** (React 18+): nhiều `setState` trong cùng một sự kiện, promise hay timeout chỉ gây **một** lần render.
- **Khác type** ở cùng vị trí trong cây (ví dụ `<div>` đổi thành `<section>`, hoặc định nghĩa component **bên trong** một component khác) thì React huỷ toàn bộ cây con và mất state. **Không bao giờ định nghĩa component bên trong component.**
- **State colocation**: đặt state ở **gần nơi dùng nhất**. State càng đặt cao trong cây thì mỗi lần đổi càng kéo nhiều component render lại.
- **Truyền children**: `<Layout>{children}</Layout>`. Khi Layout tự đổi state, `children` không render lại vì phần tử đó được tạo ở component cha.

B2. Bộ công cụ tối ưu: dùng cái gì, khi nào

Kỹ thuật	Giải quyết vấn đề gì	Lưu ý
`React.memo`	Con render lại vì cha render, dù props không đổi	Chỉ có hiệu quả khi props ổn định về tham chiếu
`useMemo`	Phép tính nặng, hoặc giữ ổn định object hay mảng truyền cho con đã memo	Không cần cho phép tính rẻ
`useCallback`	Giữ ổn định hàm truyền cho con đã memo, hoặc hàm nằm trong deps của effect	Nếu con không memo thì vô ích
Tách context hoặc dùng selector	Context đổi khiến mọi consumer render lại	Tách context theo tần suất thay đổi, hoặc dùng Zustand với selector
`useTransition` / `useDeferredValue`	Render nặng chặn việc gõ phím hay click	Không giảm số request
Virtualization	Danh sách hay bảng có hàng nghìn dòng	Cần chiều cao dòng cố định, hoặc đo được
Debounce / throttle	Sự kiện bắn liên tục (gõ phím, cuộn, resize)	Debounce: chạy sau khi ngừng. Throttle: tối đa 1 lần mỗi khoảng thời gian
Web Worker	Tính toán nặng chặn luồng chính	Dữ liệu được copy qua `postMessage`
Code splitting (`lazy` , route `lazy`)	Bundle ban đầu quá lớn	Tách theo route trước, sau đó tới các thư viện nặng (chart, editor)

> **Đo trước, tối ưu sau.** Công cụ đo: React DevTools **Profiler** (bật "Highlight updates" để thấy component nào render), tab Performance của Chrome (tìm long task trên 50ms), Lighthouse, và web-vitals. Nếu chưa đo được vấn đề thì nói "sẽ thêm `useMemo` " là câu trả lời mức junior.

B3. Core Web Vitals

Chỉ số	Đo cái gì	Mức tốt	Cách cải thiện
LCP (Largest Contentful Paint)	Thời điểm phần tử lớn nhất trong khung nhìn hiện ra	≤ 2,5s	Tối ưu ảnh chính (định dạng hiện đại, `fetchpriority="high"` , preload), giảm JS chặn render, SSR/SSG, CDN, cache

INP (Interaction to Next Paint)	Độ trễ từ lúc tương tác tới lúc trang vẽ lại	≤ 200ms	Chia nhỏ long task, dùng `useTransition`, Web Worker, giảm số render, debounce
CLS (Cumulative Layout Shift)	Mức độ bố cục bị nhảy	≤ 0,1	Khai báo kích thước ảnh và video, skeleton cùng kích thước với nội dung thật, `font-display`, không chèn nội dung lên phía trên

B4. Bundle, tài nguyên và mạng

- **Code splitting theo route**: dự án đã dùng `lazy` trong `router.tsx`. Có thể prefetch route khi user hover vào link.
- **Tree-shaking**: dùng import có tên (named import) và thư viện hỗ trợ ESM. Phân tích bundle bằng `rollup-plugin-visualizer`.
- **Ảnh**: dùng `loading="lazy"` cho ảnh nằm dưới màn hình đầu, `srcset` / `sizes`, định dạng WebP hoặc AVIF, và khai báo `width` / `height` để tránh CLS.
- **Font**: `font-display: swap`, chỉ lấy tập ký tự cần dùng (subset), preload font chính.
- **Cache HTTP**: file đã có hash trong tên thì đặt `Cache-Control: immutable`. File HTML thì không cache, hoặc cache ngắn.
- **Dữ liệu**: TanStack Query lo cache, gộp các request trùng (dedupe), `staleTime`, prefetch, optimistic update. Gọi API song song thay vì tạo "thác nước" (waterfall).

B5. Các use case hay gặp và cách giải chuẩn

Use case	Cách giải
Xác thực (access token và refresh token)	Access token giữ trong bộ nhớ, refresh token trong cookie httpOnly. Axios interceptor bắt lỗi 401 thì gọi refresh **đúng một lần**; các request khác trong lúc đó xếp hàng chờ, có token mới thì chạy lại. Đồng bộ đăng xuất giữa các tab bằng BroadcastChannel.
Bảo vệ route	`ProtectedRoute` và `GuestRoute` bọc nhóm route. Hiện loading trong lúc kiểm tra phiên, tránh chuyển hướng nhầm.
Form phức tạp	react-hook-form kết hợp zod, dùng chung schema với BE. Lỗi từ server được gắn vào đúng field (`setError`).
Lỗi khi render	Error Boundary theo route hoặc theo vùng. Lỗi khi gọi API thì xử lý qua trạng thái `isError`, kèm nút "Thử lại".
Optimistic UI (kéo thả Kanban, like)	Trong `onMutate`: lưu snapshot, rồi `setQueryData`. Nếu lỗi thì khôi phục snapshot. Trong `onSettled` thì invalidate.
Infinite scroll	`useInfiniteQuery` kết hợp `IntersectionObserver` đặt ở một phần tử cuối danh sách. Nếu danh sách dài thì virtualize.
Realtime	WebSocket hoặc SSE nhận sự kiện, sau đó `invalidateQueries` hoặc `setQueryData`. Không tạo một store song song với cache của TanStack Query.
Modal, toast, confirm toàn app	Store kết hợp một Root duy nhất và portal. Có thể dùng hàm `confirm()` trả về Promise.
Bộ lọc và phân trang	Lưu trong **URL search params**: chia sẻ được, giữ nguyên khi F5, nút back hoạt động đúng.
Upload file lớn	Xin presigned URL từ BE, upload thẳng lên S3, hiển thị tiến độ, cho phép huỷ bằng `AbortController`.
Đồng bộ nhiều tab	BroadcastChannel, sự kiện `storage`, Web Locks API.

Phần C: Cấu trúc Backend để trở thành Fullstack

C1. Cấu trúc thư mục: theo module (feature-based)

```
server/src/
├── app.ts                  # tạo express app: middleware chung, mount router, errorHandler
├── server.ts               # listen, graceful shutdown (đóng DB, server khi nhận SIGTERM)
├── config/                 # env.ts (zod validate env), database.ts (Prisma client dùng chung)
├── middlewares/            # auth, validateBody/Query/Params, errorHandler, notFound, rateLimit, requestId
├── lib/                    # AppError, jwt, password hash, logger, các tiện ích
├── types/express.d.ts      # mở rộng Request: user, membership, project...
└── modules/
    └── workspace/
        ├── workspace.routes.ts       # URL + chuỗi middleware
        ├── workspace.controller.ts   # đọc req, gọi service, trả res (mỏng)
        ├── workspace.service.ts      # business logic + truy cập DB
        ├── workspace.middleware.ts   # requireWorkspaceMember(role)
        └── workspace.test.ts         # integration test (supertest)
packages/shared/            # zod schema + type dùng chung cho FE và BE (một nguồn chuẩn duy nhất)
```

Vì sao chia theo module thay vì theo loại file (`controllers/` , `services/` ...): code của một tính năng nằm cùng một chỗ, dễ tìm, dễ xoá, dễ giao việc theo tính năng. Khi dự án lớn có thể tách tiếp thành microservice theo đúng ranh giới module.

C2. Vòng đời một request

```
Request
 → helmet · cors · express.json · cookieParser · requestId · logger      (middleware chung)
 → rateLimit (auth)                                                       (chống brute-force)
 → authenticate     : verify JWT → req.user                    (401 nếu thiếu hoặc sai)
 → validate         : zod parse body / params / query          (400 + errors theo field)
 → authorize        : requireWorkspaceMember(role)             (404 người ngoài · 403 sai role)
 → controller       : lấy dữ liệu đã validate, gọi service, trả status + JSON
 → service          : business rule, transaction, Prisma
 → (throw AppError) → errorHandler: format lỗi thống nhất, log, ẩn stack ở production
```

Tầng	Được làm	Không được làm
Route	Khai báo URL và thứ tự middleware	Chứa logic
Controller	Đọc `req` , gọi service, chọn status code	Truy vấn DB, chứa business rule
Service	Business logic, transaction, gọi DB	Đụng tới `req` / `res` (để còn test được và dùng lại ở job hay socket)
Middleware	Các việc dùng chung: xác thực, phân quyền, validate	Logic riêng của một endpoint

C3. Kiến thức BE cần nắm: checklist kèm vị trí trong dự án

Chủ đề	Cần biết	Dự án
Node.js runtime	Event loop (call stack, microtask, macrotask), non-blocking I/O. Việc nặng CPU sẽ chặn mọi request, nên phải đưa sang `worker_threads` hoặc queue. Stream khi xử lý file lớn.	
Thiết kế REST	Danh từ số nhiều, resource lồng nhau, đúng status code (200, 201, 204, 400, 401, 403, 404, 409, 422, 500), phân trang (offset hoặc cursor), lọc và sắp xếp, versioning `/api/v1` , idempotency (PUT, DELETE, Idempotency-Key)	Task 6–9
Validation	Validate ở biên: body, params, query, env. Dùng `z.coerce` cho params, vì zod tự loại bỏ field lạ nên chống được mass assignment.	`@ttm/shared`

Xử lý lỗi	Dùng class `AppError`, một errorHandler duy nhất, format thống nhất `{ message, errors }`. Express 5 tự bắt lỗi trong hàm async.	Đã có
Xác thực (AuthN)	Hash mật khẩu bằng bcrypt hoặc argon2. JWT access token ngắn hạn, refresh token xoay vòng (rotation) và thu hồi được (lưu trong DB). So sánh JWT với session.	Task 1
Phân quyền (AuthZ)	RBAC, kiểm tra quyền sở hữu. **IDOR**: luôn lấy danh tính từ token và kiểm tra quan hệ cha trên URL. Trả 404 hay 403?	`requireWorkspaceMember`
Database	Mô hình hoá quan hệ, khoá ngoại, index (composite, quy tắc leftmost prefix), transaction (nested write, `$transaction`), N+1 (giải bằng `_count`, `groupBy`, `include`), xoá mềm, migration	Task 6–7
Hiệu năng	Chỉ `select` cột cần, phân trang, index, cache (Redis, cache-aside, TTL, invalidation), connection pool, nén response	Task 11
Realtime	WebSocket hoặc Socket.IO, room, xác thực khi handshake, Redis adapter khi scale nhiều instance	Task 10
Background job	Queue (BullMQ + Redis) cho gửi email, xuất file, thông báo. Retry có backoff, job idempotent.	Mở rộng
Bảo mật	OWASP Top 10: injection (ORM đã tham số hoá), XSS, CSRF (SameSite), cấu hình CORS, helmet, rate limit, giới hạn kích thước body, không log secret	Task 11
Logging và giám sát	Log có cấu trúc (pino), request id, log level, health check `/health`, metrics, tracing (OpenTelemetry)	Task 11
Testing	Integration test (supertest với DB test thật), unit test cho logic thuần, test các trường hợp phân quyền	Task 5
Deploy	Docker (multi-stage build), biến môi trường và secret, `migrate deploy` khi release, CI/CD, graceful shutdown, server stateless để scale ngang	Task 11

C4. Lộ trình trở thành Fullstack (từ nền tảng FE)

Giai đoạn	Mục tiêu	Bằng chứng để nói khi phỏng vấn
1. API cơ bản	Express, routing, validation, error handling, CRUD với Prisma	Workspace, Project, Board API (Task 6–7) ✅
2. Auth và phân quyền	JWT kèm refresh rotation, cookie httpOnly, RBAC, chống IDOR	Task 1 và middleware phân quyền ✅
3. Database sâu hơn	Index, transaction, N+1, xoá mềm, đọc `EXPLAIN ANALYZE`	`taskCounts` dùng `groupBy` thay vì N+1
4. Chất lượng	Integration test, CI	Task 5: DB test riêng, MSW, GitHub Actions ✅
5. Realtime và bất đồng bộ	WebSocket, optimistic UI, queue	Task 9–10 (Task, Kanban)
6. Production	Bảo mật, logging, Docker, deploy, giám sát	Task 11
7. System design	Cache, scale ngang, message queue, CDN, các trade-off của định lý CAP	Tự luyện vẽ: "Thiết kế Trello", "Thiết kế bảng giá chứng khoán"

C5. Câu hỏi BE hay gặp (trả lời nhanh)

Câu hỏi	Ý chính
Event loop của Node chạy thế nào?	Một luồng chạy JS. I/O được giao cho libuv. Sau mỗi tác vụ, chạy hết microtask (Promise) rồi mới tới macrotask (timer, I/O). Code đồng bộ nặng sẽ chặn mọi request.
JWT hay session?	JWT không cần lưu trạng thái, dễ scale, nhưng khó thu hồi, nên thường dùng access token ngắn hạn kèm refresh token lưu DB. Session thu hồi dễ, nhưng cần một store dùng chung khi có nhiều server.

N+1 query là gì?	1 query lấy danh sách, rồi N query lấy dữ liệu cho từng dòng. Giải bằng join hoặc `include`, `_count`, `groupBy`, hoặc DataLoader. Dự án: 50 workspace từng tốn 51 query.
Khi nào cần transaction?	Khi nhiều thao tác ghi phải cùng thành công hoặc cùng thất bại. Ví dụ tạo workspace kèm member ADMIN và project đầu tiên.
Index hoạt động thế nào? Khi nào không nên đánh?	B-tree giúp tìm kiếm với độ phức tạp O(log n). Đổi lại, mỗi lần ghi chậm hơn và tốn dung lượng. Composite index tuân theo quy tắc leftmost prefix. Không cần đánh index cho cột có ít giá trị khác nhau, hoặc bảng rất nhỏ.
CORS là gì?	Cơ chế của trình duyệt: server khai báo những origin được phép gọi tới mình. Request phức tạp sẽ có preflight `OPTIONS`. Nếu dùng cookie thì cần `credentials: true` và một origin cụ thể (không dùng `*`).
Trả 401, 403 hay 404?	401: chưa xác thực. 403: đã xác thực nhưng không đủ quyền. 404: không tồn tại, hoặc **muốn che giấu** sự tồn tại với người ngoài.
Idempotency là gì?	Gọi nhiều lần cho cùng một kết quả. GET, PUT, DELETE là idempotent, POST thì không. Với thanh toán, dùng header `Idempotency-Key` để chống trừ tiền 2 lần khi client gửi lại request.